CFD 2025-26 – Lab 02 – solution

The full code for this lab is provided in the notebook `CFDlab02.ipynb`.

Main identities and definitions

Integration by parts

$$\int_\Omega (-\Delta \boldsymbol{u}) \cdot \boldsymbol{v}\, d\boldsymbol{x} = \int_\Omega \nabla \boldsymbol{u}) : \nabla \boldsymbol{v}\, d\boldsymbol{x} - \int_{\partial\Omega} (\nabla \boldsymbol{u})\boldsymbol{n} \cdot \boldsymbol{v}\, ds \tag{1}$$

$$\int_\Omega \nabla q \cdot \boldsymbol{v}\, d\boldsymbol{x} = -\int_\Omega q \operatorname{div} \boldsymbol{v}\, d\boldsymbol{x} - \int_{\partial\Omega} q\boldsymbol{n} \cdot \boldsymbol{v}\, ds \tag{2}$$

Norms

$$\|\boldsymbol{u}\|_{L^2} = \left(\int_\Omega |\boldsymbol{u}|^2 d\boldsymbol{x}\right)^{\frac{1}{2}}, \quad \|\boldsymbol{u}\|_{H^1_0} = \left(\int_\Omega |\nabla \boldsymbol{u}|^2 d\boldsymbol{x}\right)^{\frac{1}{2}}, \quad \|\boldsymbol{u}\|_{L^2} = \left(\|\boldsymbol{u}\|^2_{L^2} + \|\boldsymbol{u}\|^2_{H^1_0}\right)^{\frac{1}{2}}. \tag{3}$$

Function spaces

$$L^2(\Omega) = \{\zeta : \Omega \to \mathbb{R} \text{ s.t. } \|\zeta\|_{L^2} < \infty\}, \qquad H^1(\Omega) = \{\zeta : \Omega \to \mathbb{R} \text{ s.t. } \|\zeta\|_{H^1} < \infty\},$$
$$H^1_\Gamma(\Omega) = \{\zeta \in H^1(\Omega) \text{ s.t. } \zeta = 0 \text{ on } \Gamma\}, \qquad H^1_0(\Omega) = H^1_{\partial\Omega}(\Omega).$$

Exercise 1

1. In order to verify that the given flow $(\boldsymbol{u}, p)$ is indeed a solution of the problem, we simply need to show that all the equations and conditions are fulfilled. This can be done straightforwardly, by noticing that $\nabla \boldsymbol{u} = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ and that $\boldsymbol{n}|_{\Gamma_3} = \pm\boldsymbol{i} = \pm \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Furthermore, the well-posedness of the problem ensures that this solution is unique.

2. Following the same steps of the previous lab session, we can write the weak formulation of the Stokes problem. To ease the notation, let $\Gamma_{\mathrm{D}} := \Gamma_{\mathrm{D1}} \cup \Gamma_{\mathrm{D2}}$ and $\boldsymbol{g}$ a function encoding the boundary data. We have to introduce the following trial and test spaces for the velocity *vectorial* field:

$$V_{\mathrm{D}} = \{\boldsymbol{v} \in [H^1(\Omega)]^2 : \boldsymbol{v} = \boldsymbol{g} \text{ on } \Gamma_{\mathrm{D}}\}, \qquad V_0 = [H^1_{\Gamma_{\mathrm{D}}}(\Omega)]^2,$$

whereas the *scalar* pressure space is

$$Q := L^2(\Omega)$$

We test the momentum equation against a generic test function $\boldsymbol{v} \in V_0$, and we use the integration by parts formulas (1)-(2) and the boundary conditions, thus obtaining

$$\int_\Omega \nabla \boldsymbol{u} \colon \nabla \boldsymbol{v} \, d\boldsymbol{x} - \int_\Omega p \operatorname{div} \boldsymbol{v} \, d\boldsymbol{x} - \int_{\partial\Omega} (\nabla \boldsymbol{u} - pI)\boldsymbol{n} \cdot \boldsymbol{v} \, ds = 0$$

$$\Updownarrow$$

$$\int_\Omega \nabla \boldsymbol{u} \colon \nabla \boldsymbol{v} \, d\boldsymbol{x} - \int_\Omega p \operatorname{div} \boldsymbol{v} \, d\boldsymbol{x} - \int_{\Gamma_D} (\nabla \boldsymbol{u} - pI)\boldsymbol{n} \cdot \cancelto{\boldsymbol{0}}{\boldsymbol{v}} ds = 0 - \int_{\partial\Omega} \cancel{(\nabla \boldsymbol{u} - pI)\boldsymbol{n}} \cdot \cancelto{\boldsymbol{0}}{\boldsymbol{v}} \, ds = 0$$

$$\Downarrow$$

$$\int_\Omega \nabla \boldsymbol{u} \colon \nabla \boldsymbol{v} \, d\boldsymbol{x} - \int_\Omega p \operatorname{div} \boldsymbol{v} \, d\boldsymbol{x} = 0$$

Testing the continuity equation against a generic test function $q \in Q$, we can write the following system:

$$\begin{cases} \int_\Omega (\nabla \boldsymbol{u} \colon \nabla \boldsymbol{v} \, d\boldsymbol{x} - p \operatorname{div} \boldsymbol{v}) d\boldsymbol{x} = 0 & \forall \boldsymbol{v} \in V_0, \\ \int_\Omega \operatorname{div} \boldsymbol{u} \; q \, d\boldsymbol{x} = 0 & \forall q \in Q. \end{cases}$$

Thanks to the arbitrariness of the test functions $\boldsymbol{v} \in V_0$ and $q \in Q$, we can combine the two equations of this system to obtain the following equivalent weak formulation:

$$\begin{gathered} \text{Find } \boldsymbol{u} = \check{\boldsymbol{u}} + \mathcal{R}\boldsymbol{g} \in V_{\mathrm{D}}, \text{ with } \check{\boldsymbol{u}} \in V_0, \text{ and } p \in Q \text{ s.t.} \\ a((\check{\boldsymbol{u}}, p), (\boldsymbol{v}, q)) = -a((\mathcal{R}\boldsymbol{g}, 0), (\boldsymbol{v}, q)), \text{ for all } (\boldsymbol{v}, q) \in V_0 \times Q, \end{gathered}$$

where

$$a((\boldsymbol{u}, p), (\boldsymbol{v}, q)) := \int_\Omega (\nabla \boldsymbol{u} : \nabla \boldsymbol{v} - p \operatorname{div} \boldsymbol{v} + \operatorname{div} \boldsymbol{u} \; q) \; d\boldsymbol{x}.$$

3 To implement the finite element approximation of the problem in `Firedrake`, we start from the mesh generation. We employ the function `RectangleMesh`:

```
n = 10
mesh = RectangleMesh(3*n, n, 3, 1)
```

Then we create the velocity and pressure finite elements spaces V and Q respectively, as well as their product $W = V \times Q$:

```
V = VectorFunctionSpace(mesh, 'P', 1)
Q = FunctionSpace(mesh, 'P', 1)
W = MixedFunctionSpace([V, Q])
```

We instantiate the trial and test functions starting from the mixed space W:

```
u, p = TrialFunctions(W)  # notice that it is PLURAL
v, q = TestFunctions(W)   # notice that it is PLURAL
```

For the source term and boundary values, we use `Constants`:

```
f = Constant((0.,0.))
g3 = Constant((0.,0.))
g4 = Constant((1.,0.))
```

The definition of bilinear forms and functionals for vector-valued functions is anlogous to the case of scalar functions, properly using the pre-defined differential operators. Notice that the `inner` command can compute the inner product either between tensors or betwen vectors.

```
a = inner(grad(u), grad(v)) * dx - div(v) * p * dx + q * div(u) * dx
L = inner(Constant((0.0,0.0)), v) * dx
```

The problem is then solved *monolithically*, that is in the complete space W:

```
wh = Function(W)
solve(a == L, wh, bcs=bcs)
uh, ph = wh.subfunctions  # extract velocity and pressure components
```

4 By changing the definition of the FE spaces `V,Q` (and consequently `W`), the different choices can be inspected. For the $\mathbb{P}^1/\mathbb{P}^0$ case, we point out that a *discontinuous* space (of element-wise constant functions) has to be defined:

```
Q = FunctionSpace(mesh, 'DP', 0)  # DP -> DISCONTINUOUS elements
```

In the case of bubble-enriched space, we need a more complex definition of the space. In Firedrake, a `FunctionSpace` is actually defined by a mesh and an object **`FiniteElement`** (created implicitly inside the `FunctionSpace` constructor used so far), which represents a finite element over a reference element (e.g. the triangle $\{(x, y)\colon x, y \in (0, 1), x + y \geq 1\}$). Bubble-enrichment can be done at the **`FiniteElement`** level, and only after that we can construct the `FunctionSpace` (using a different constructor), as follows:

```
V1_el = FiniteElement('CG', mesh.ufl_cell(), 1)
B_el = FiniteElement('Bubble', mesh.ufl_cell(),
                     mesh.topological_dimension() + 1)
V_el = VectorElement(NodalEnrichedElement(V1_el, B_el))

V = FunctionSpace(mesh, V_el)
```

The results for different FE pairs are displayed in Figure 1.

We can observe that spurious oscillations in the pressure field appear throughout the whole domain when the unstable FE pairs $\mathbb{P}^1/\mathbb{P}^0$ or $\mathbb{P}^1/\mathbb{P}^1$ are chosen, whereas they do not arise if the stable pairs $\mathbb{P}^1_b/\mathbb{P}^1$ and $\mathbb{P}^2/\mathbb{P}^1$ are considered (the magnitude of dishomogeneities is comparable with machine precision). Actually, in some cases (possibly depending on the specific problems, the boundary conditions, the machine on which the solver is run, ...) the solution in the case of unstable FE spaces may not be found, with Firedrake returning the following error:

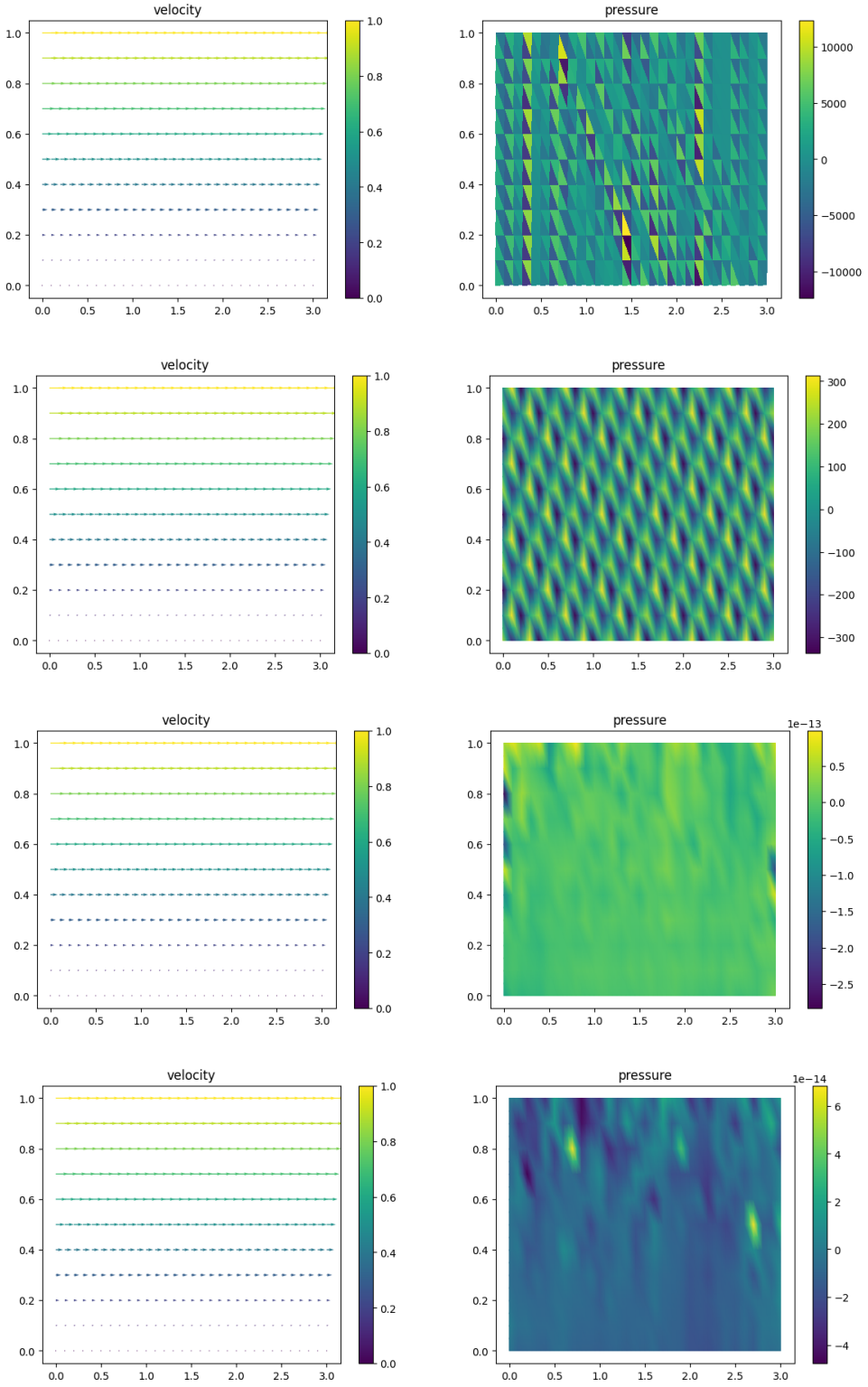

Figure 1: Velocity and pressure fields for the FE pairs (from top to bottom) $\mathbb{P}^1/\mathbb{P}^0$, $\mathbb{P}^1/\mathbb{P}^1$, $\mathbb{P}^1_b/\mathbb{P}^1$, and $\mathbb{P}^2/\mathbb{P}^1$.

```
ConvergenceError: Nonlinear solve failed to converge after 0 nonlinear
    iterations.
Reason:
   DIVERGED_LINEAR_SOLVE
```

i.e. telling that the solution of the linear system is not possible. Indeed, when unstable FE spaces are used, the linear system associated to the FE method is singular: the fact that in some cases we find a solution is basically just "luck" (rigorously, the right-hand-side vector belongs to the (non-full-rank) column space of the system matrix).

We also notice that the different FE spaces qualitatively provide the same velocity field (solution accuracy is going to be discussed in the following point). Indeed, we know from the theory of Stokes problem that the choice of the spaces affects the discrete inf-sup condition, which in turn is related to the uniqueness of just the *pressure* field.

Note: From the plots of Figure 1, it may seem that the same dof's are considered for velocity in all cases. This is because the library `matplotlib` natively plots only $\mathbb{P}^1$ functions: in the $\mathbb{P}^2$ and $\mathbb{P}^1_b$ cases, it interpolates the solution on the $\mathbb{P}^1$ space and then shows the same number of arrows as in the other ones.

5 The total number of degrees of freedom N_{dof} corresponding to each choice can be obtained by `W.dim()` (or equivalently by `V.dim()+Q.dim()`).

```
print('Ndofs - velocity :',V.dim(),', pressure :',Q.dim(),
      ', total :',W.dim())
```

Denoting by N_{v} and N_{el} the number of vertices and elements of mesh, N_{dof} can actually be computed a priori in terms of $N_{\text{v}}, N_{\text{el}}$: for example, for the structured rectangular mesh that we consider, with $n = 10$ elements for each unit of space, $N_{\text{v}} = (n+1)(3n+1) = 341$, $N_{\text{el}} = 2(n \times 3n) = 600$, and thus $N_{\text{dof}}(\mathbb{P}^1/\mathbb{P}^1) = 2N_{\text{v}} + N_{\text{v}} = 1023$, $N_{\text{dof}}(\mathbb{P}^1/\mathbb{P}^0) = 2N_{\text{v}} + N_{\text{el}} = 1282$.

The errors are defined in terms of the norms (3)

$$\|\boldsymbol{u} - \boldsymbol{u}_h\|_{L^2} = \left(\int_\Omega |\boldsymbol{u} - \boldsymbol{u}_h|^2 \, d\boldsymbol{x}\right)^{1/2}, \qquad \|\nabla \boldsymbol{u} - \nabla \boldsymbol{u}_h\|_{L^2} = \left(\int_\Omega |\nabla \boldsymbol{u} - \nabla \boldsymbol{u}_h|^2 \, d\boldsymbol{x}\right)^{1/2},$$

$$\|p - p_h\|_{L^2} = \left(\int_\Omega |p - p_h|^2 \, d\boldsymbol{x}\right)^{1/2}.$$

To compute them, we can use the definition: With the type `func`, one can quickly define functions of x and y (with this syntax only functions of this type are available).

```
x = SpatialCoordinate(mesh)
u_ex = as_vector([x[1],0.])
grad_u_ex = as_tensor([[0.,1.],[0.,0.]])
p_ex = Constant(0.)
errL2u = sqrt(assemble( inner(uh-u_ex,uh-u_ex) * dx ))
errH10u = sqrt(assemble(
          inner(grad(uh)-grad_u_ex,grad(uh)-grad_u_ex) * dx ))
errL2p = sqrt(assemble( inner(ph-p_ex,ph-p_ex) * dx ))
```

```
print('Errors - L2-u:', errL2u, ', H10-u:', errH10u,
      ', L2-p:', errL2p)
```

Since all the the integrands are known functions (either the exact solution or the numerical approximation we have already computed), the `assemble` command takes the given for and returns the scalar number resulting from the integration. Alternatively, we can use Firedrake's **`errornorm`** functions, as follows

```
errL2u = errornorm(u_ex, uh, 'L2')
errL2p = errornorm(p_ex, ph, 'L2')
print('Errors - L2-u:', errL2u, ', L2-p:', errL2p)
# errornorm does not have the 'H1' seminorm (H10),
# but only the full H1 norm
errH1u = errornorm(u_ex, uh, 'H1')
print('Errors - H1-u:', errH1u)
```

The resulting errors are as follows:

	N_{dofs}	$\|\boldsymbol{u}-\boldsymbol{u}_h\|_{L^2}$	$\|\nabla\boldsymbol{u}-\nabla\boldsymbol{u}_h\|_{L^2}$	$\|p-p_h\|_{L^2}$
$\mathbb{P}^1/\mathbb{P}^0$	1282	$2.169\cdot10^{-13}$	$1.616\cdot10^{-12}$	6876
$\mathbb{P}^1/\mathbb{P}^1$	1023	$5.276\cdot10^{-15}$	$6.044\cdot10^{-14}$	204.01
$\mathbb{P}^1_b/\mathbb{P}^1$	2223	$3.343\cdot10^{-15}$	$2.715\cdot10^{-14}$	$3.298\cdot10^{-14}$
$\mathbb{P}^2/\mathbb{P}^1$	2903	$3.157\cdot10^{-15}$	$1.974\cdot10^{-14}$	$2.110\cdot10^{-14}$

This table confirms what observed at the previous point: if an unstable FE pair is considered, a large error is introduced in the pressure component; regarding velocity, instead, negligible errors can be observed for any choice of the FE pair.

Exercise 2

1. We introduce the spaces

$$V_0 = [H^1_{\Gamma_D}(\Omega)]^2, \qquad V_D = \left\{\boldsymbol{v}\in[H^1(\Omega)]^2 : \boldsymbol{u}|_{\Gamma_{\text{wall}}} = \boldsymbol{0}, \boldsymbol{u}|_{\Gamma_{\text{in}}} = (1-y^2)\boldsymbol{i}\right\}, \qquad Q = L^2(\Omega),$$

and, following the same steps of the previous exercises (paying attention to the constant viscosity μ multiplying the diffusion term), we can obtain the following weak formulation:

$$\text{Find } (\boldsymbol{u},p)\in V_D\times Q, \text{ with } \boldsymbol{u}=\check{\boldsymbol{u}}+\mathcal{R}\boldsymbol{g}, \text{ such that } \check{\boldsymbol{u}}\in V_0 \text{ and}$$
$$a((\check{\boldsymbol{u}},p),(\boldsymbol{v},q)) = \widetilde{B}_c(\boldsymbol{v}), \text{ for all } (\boldsymbol{v},q)\in V_0\times Q,$$

where

$$a((\boldsymbol{u},p),(\boldsymbol{v},q)) = \int_\Omega (\mu\nabla\boldsymbol{u}:\nabla\boldsymbol{v} - p\,\text{div}\,\boldsymbol{v} - \text{div}\,u\,q)\,d\boldsymbol{x},$$
$$\widetilde{B}_c(\boldsymbol{v}) = -a((\mathcal{R}\boldsymbol{g}_D,0),(\boldsymbol{v},q)),$$

where $\boldsymbol{g}_D$ collectively denotes the Dirichlet data and $\mathcal{R}\boldsymbol{g}_D$ its lifting.

2 We introduce a mesh $\mathcal{T}_h$, triangulating the domain Ω, and define the following finite element spaces over it:

$$X_h^r = \left\{\zeta \in C^0\left(\overline{\Omega}\right) : \zeta|_K \in \mathbb{P}^r(K)\ \forall K \in \mathcal{T}_h\right\},$$
$$V_{h,D} = [X_h^2]^2 \cap V_D, \qquad V_h = [X_h^2]^2 \cap V_0, \qquad Q_h = X_h^1.$$

Then, the finite element method for the problem at hand reads as follows:

$$\begin{gathered}\text{Find } (\boldsymbol{u}_h, p_h) \in V_{h,\mathrm{D}} \times Q_h, \text{ with } \boldsymbol{u}_h = \check{\boldsymbol{u}}_h + \mathcal{R}_h \boldsymbol{g}_\mathrm{D}, \text{ such that } \check{\boldsymbol{u}}_h \in V_h \text{ and} \\ a((\check{\boldsymbol{u}}_h, p_h), (\boldsymbol{v}_h, q_h)) = B_c(\boldsymbol{v}_h), \text{ for all } (\boldsymbol{v}_h, q_h) \in V_h \times Q_h,\end{gathered} \tag{4}$$

where B_c is analogous to $\widetilde{B}_c$ with the *discrete* lifting $\mathcal{R}_h \boldsymbol{g}_\mathrm{D}$ of the Dirichlet data, instead of the continuous one.

3 We introduce the algebraic formulation of the finite element mehtod (4). We denote by $\{\boldsymbol{\phi}_j\}_{j=1}^{\mathcal{N}_{\boldsymbol{u}}}$ and $\{\psi_l\}_{l=1}^{\mathcal{N}_p}$ the bases of the spaces V_h and Q_h, respectively. The discrete solution can then be written as

$$\boldsymbol{u}_h = \sum_{j=1}^{\mathcal{N}_{\boldsymbol{u}}} u_j \boldsymbol{\phi}_j, \qquad p_h = \sum_{l=1}^{\mathcal{N}_p} p_l \psi_l,$$

and the unknown coefficients $u_j, j = 1, \dots \mathcal{N}_{\boldsymbol{u}}$, $p_l, l = 1, \dots, \mathcal{N}_p$ can be stored in vectors $\boldsymbol{U} \in \mathbb{R}^{\mathcal{N}_{\boldsymbol{u}}}, \boldsymbol{P} \in \mathbb{R}^{\mathcal{N}_p}$. Moreover, the finite element method can be reformulated with the test functions $\boldsymbol{v}_h, p_h$ ranging among the basis functions, thus obtaining the following formulations, both equivalent to (4):

- find $\boldsymbol{U} \in \mathbb{R}^{\mathcal{N}_{\boldsymbol{u}}}, \boldsymbol{P} \in \mathbb{R}^{\mathcal{N}_p}$ s.t. $$a\left(\left(\sum_{j=1}^{\mathcal{N}_{\boldsymbol{u}}} u_j \boldsymbol{\phi}_j, \sum_{l=1}^{\mathcal{N}_p} p_l \psi_l\right), (\boldsymbol{\phi}_i, \psi_k)\right) = B_c(\boldsymbol{\phi}_i),$$ for all $, i = 1, \dots \mathcal{N}_{\boldsymbol{u}}, k = 1, \dots, \mathcal{N}_p$;
- find $\boldsymbol{U} \in \mathbb{R}^{\mathcal{N}_{\boldsymbol{u}}}, \boldsymbol{P} \in \mathbb{R}^{\mathcal{N}_p}$ s.t. $$\Sigma \begin{bmatrix}\boldsymbol{U} \\ \boldsymbol{P}\end{bmatrix} = \begin{bmatrix}\boldsymbol{F} \\ \boldsymbol{0}\end{bmatrix},$$

where a proper definition of the matrix $\Sigma \in \mathbb{R}^{(\mathcal{N}_{\boldsymbol{u}}+\mathcal{N}_p)\times(\mathcal{N}_{\boldsymbol{u}}+\mathcal{N}_p)}$ and of the vector $\boldsymbol{F} \in \mathbb{R}^{\mathcal{N}_{\boldsymbol{u}}}$ is going to be provided in the following point.

We now pass to the implementation of the problem. For the mesh we use the `RectangleMesh` command similarly to Exercise 1, using also the **`originY`** option:

```
mesh = RectangleMesh(2*n, 2*n, 2., 1., originY=-1.)
```

Notice that the mesh size is $h = 1/\mathtt{n}$.

The problem can be defined by the bilinear form and functional as in the previous exercises, and different boundary conditions are created on different boundaries:

```
f = Constant((0.,0.))
a = 1.e-2*inner(grad(u), grad(v)) * dx - div(v) * p * dx + q * div(u)
    * dx
L = dot(f, v) * dx
```

```
x = SpatialCoordinate(mesh)
gD = (1.-x[1]**2, 0.)
bc_in = DirichletBC(W.sub(0), gD, 1)
bc_wall = DirichletBC(W.sub(0), Constant((0.,0.)), (3, 4))
bcs = (bc_in, bc_wall)
```

We then collect all this information into a `LinearVariationalProblem`

```
w = Function(W)
vpb = LinearVariationalProblem(a, L, w, bcs)
```

We can inspect the matrix Σ and vector $\boldsymbol{b}$ by assembling them explicitly with the `assemble` command. In terms of Dirichlet boundary conditons, we can see how both the matrix and vector are affected by their imposition, comparing the results of

```
Sigma = assemble(a)                b = assemble(L)
```

and

```
Sigma = assemble(a, bcs=bcs)       b = assemble(L, bcs=bcs)
```

Specifically, with Dirichlet BC's, the rows and columns of Σassociated to Dirichlet dof's are replaced by corresponding rows and columns of the identity matrix, and the related elements of $\boldsymbol{b}$ are set to the values of the Dirichlet data (cf. **`CFDlab02.ipynb`**).

4. For the solution of the problem, we use

```
solver =  LinearVariationalSolver(vpb, solver_parameters=parameters)
```

where **`vpb`** is the `LinearVariationalProblem` defined at the previous point and the parameters of the solver are set up as follows:

```
parameters = {'ksp_type': 'preonly',
              'pc_type': 'lu',
              'pc_factor_mat_solver_type': 'mumps'}
```

Further information on the KSP solver and preconditioning will be the topic of the next lab session. For the moment, we can already say that `'preonly'` instructs Firedrake to perform only the preconditioning of the system, which can be used to actually apply the *direct* solver specified as **`pc_type`**. Regarding the specification `'pc_factor_mat_solver_type': 'mumps'`, it selects the linear algebra software library employed in the application of the preconditioner/direct solver: in this case it is MUMPS (**`https://mumps-solver.org`**).

We can now solve the problem by calling **`solver.solve()`**. To have also a measure of the computational time required by the solution, we use the **`perf_counter`** function of the **`time`** python module, which returns the current time whenever it is called:

```
t0 = perf_counter()
```

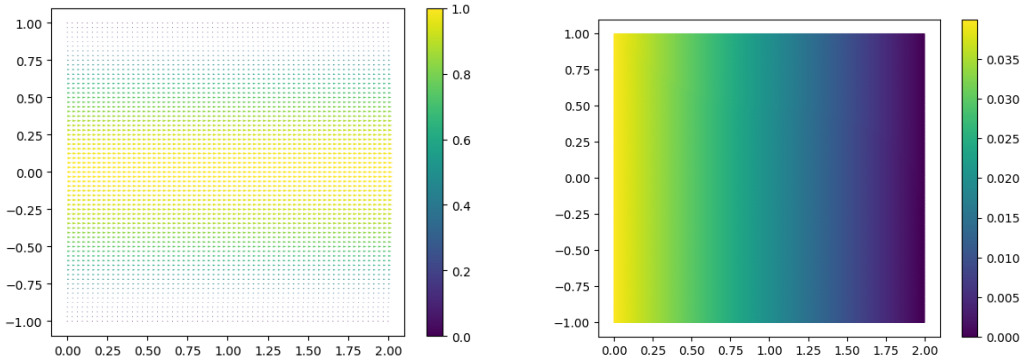

Figure 2: Exercise 3. Velocity and pressure field for the FE pair $\mathbb{P}^2/\mathbb{P}^1$ with $h = 0.03125$.

```
solver.solve()
print('elapsed time = ', perf_counter() - t0,
      's    -    # iter = ', solver.snes.ksp.getIterationNumber())
```

N.B. The actual elapsed time may vary very much depending on the different machines and current overall load during the execution: however, the observations reported here (if not the specific number of seconds) should hold.

We obtain the velocity and pressure plots reported in Figure 2. We can notice that the problem under investigation models a 2D Poiseuille flow in a square pipe, where the left edge is the inlet (on which a parabolic profile is imposed), the right edge is the outlet, and the top and bottom boundaries are the pipe wall, where the so-called *no-slip* condition is imposed. In such settings, the velocity profile is parabolic in each vertical section of the pipe, and the pressure decays linearly along the x direction: indeed, the exact solution for the domain Ω at hand is

$$\boldsymbol{u}(x, y) = (1 - y^2)\boldsymbol{i}, \qquad p(x, y) = 2\mu(2 - x).$$

5 Regarding the effects of considering different values of the viscosity, we can make two comments at this point of the course. These considerations are going to be discussed more extensively when the *Navier*-Stokes equations will be addressed.

Looking at the solution (in the resulting plots and/or in the exact expressions reported in the previous point) we can notice that velocity is unaffected by viscosity variations, because we always impose the same profile at the inlet, with the same flowrate, whereas the pressure drop between inlet and outlet increases with the viscosity. From a physical viewpoint, this means that more viscous fluid require more energy (represented by the pressure drop) in order to flow with the same flowrate through the same pipe.